Name: K.Likitha

Hall Ticket: 2303A52144

Batch: 41

Task 1: Sorting Student Records for Placement Drive

Prompt:

Create a Python program to store student records (Name, Roll, CGPA) and sort them in descending CGPA using Quick Sort and Merge Sort. Measure runtime for large datasets and add a function to display top 10 students.

Code:

```
import random
import time

# 1. Define Student class
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"Student(Name: {self.name}, Roll: {self.roll_number}, CGPA:
{self.cgpa:.2f})"

# 2. Implement function to generate student data
def generate_students(num_students):
    students = []
    for i in range(num_students):
        name = f"Student_{i+1}"
        roll_number = i + 1
        cgpa = round(random.uniform(0.0, 10.0), 2)
        students.append(Student(name, roll_number, cgpa))
    return students

# 3. Implement Quick Sort (descending by CGPA)
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2]
    left = [s for s in students if s.cgpa > pivot.cgpa]
    middle = [s for s in students if s.cgpa == pivot.cgpa]
    right = [s for s in students if s.cgpa < pivot.cgpa]
    return quick_sort(left) + middle + quick_sort(right)

# 4. Implement Merge Sort (descending by CGPA)
def merge_sort(students):
    if len(students) <= 1:
        return students

    mid = len(students) // 2
    left_half = students[:mid]
```

```
    right_half = students[mid:]

    left_half = merge_sort(left_half)
    right_half = merge_sort(right_half)

    return merge(left_half, right_half)

def merge(left, right):
    merged_list = []
    left_idx, right_idx = 0, 0

    while left_idx < len(left) and right_idx < len(right):
        if left[left_idx].cgpa >= right[right_idx].cgpa:
            merged_list.append(left[left_idx])
            left_idx += 1
        else:
            merged_list.append(right[right_idx])
            right_idx += 1

    merged_list.extend(left[left_idx:])
    merged_list.extend(right[right_idx:])
    return merged_list

# 5. Implement function to measure performance
def measure_performance(sort_func, students_data):
    start_time = time.time()
    sorted_students = sort_func(students_data.copy()) # Use a copy to avoid modifying
original list
    end_time = time.time()
    return sorted_students, (end_time - start_time) * 1000 # Time in milliseconds

# 6. Implement function to display top 10 students
def display_top_10_students(students):
    if not students:
        print("No students to display.")
        return

    print("\n--- Top 10 Students ---")
    for i, student in enumerate(students[:10]):
        print(f"{i+1}. {student}")
    print("-----------------------")

# 7. Main program loop (User Interface)
def main():
    student_records = None
    sorted_records = None
    current_sort_method = None

    while True:
        print("\n--- Student Records Program ---")
        print("1. Generate Student Data")
        print("2. Sort using Quick Sort and display performance")
        print("3. Sort using Merge Sort and display performance")
        print("4. Display Top 10 Students")
```

```
        print("5. Exit Program")
        choice = input("Enter your choice: ")

        if choice == '1':
            try:
                num_students = int(input("Enter the number of students to generate: "))
                if num_students <= 0:
                    raise ValueError
                print(f"Generating {num_students} students...")
                student_records = generate_students(num_students)
                sorted_records = None # Reset sorted data after new generation
                current_sort_method = None
                print("Student data generated successfully.")
            except ValueError:
                print("Invalid input. Please enter a positive integer for the number of
students.")
        elif choice == '2':
            if student_records is None:
                print("Please generate student data first (Option 1).")
                continue
            print("Sorting using Quick Sort...")
            sorted_records, time_taken = measure_performance(quick_sort, student_records)
            current_sort_method = "Quick Sort"
            print(f"Quick Sort completed in {time_taken:.2f} ms.") # Re-added performance
display
        elif choice == '3':
            if student_records is None:
                print("Please generate student data first (Option 1).")
                continue
            print("Sorting using Merge Sort...")
            sorted_records, time_taken = measure_performance(merge_sort, student_records)
            current_sort_method = "Merge Sort"
            print(f"Merge Sort completed in {time_taken:.2f} ms.")
        elif choice == '4':
            if sorted_records is None:
                if student_records is None:
                    print("Please generate and then sort student data first (Options 1,
then 2 or 3).")
                else:
                    print("Please sort the student data first (Options 2 or 3).")
                continue
            print(f"Displaying top 10 students after {current_sort_method}:")
            display_top_10_students(sorted_records)
        elif choice == '5':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 5.")

if __name__ == "__main__":
    main()
```

Output:

```
--- Student Records Program ---
1. Generate Student Data
2. Sort using Quick Sort and display performance
3. Sort using Merge Sort and display performance
4. Display Top 10 Students
5. Exit Program
Enter your choice: 1
Enter the number of students to generate: 30
Generating 30 students...
Student data generated successfully.

--- Student Records Program ---
1. Generate Student Data
2. Sort using Quick Sort and display performance
3. Sort using Merge Sort and display performance
4. Display Top 10 Students
5. Exit Program
Enter your choice: 2
Sorting using Quick Sort...
Quick Sort completed in 0.05 ms.

--- Student Records Program ---
1. Generate Student Data
2. Sort using Quick Sort and display performance
3. Sort using Merge Sort and display performance
4. Display Top 10 Students
5. Exit Program
Enter your choice: 3
Sorting using Merge Sort...
Merge Sort completed in 0.09 ms.

--- Student Records Program ---
1. Generate Student Data
2. Sort using Quick Sort and display performance
3. Sort using Merge Sort and display performance
4. Display Top 10 Students
5. Exit Program
```

```
--- Student Records Program ---
1. Generate Student Data
2. Sort using Quick Sort and display performance
3. Sort using Merge Sort and display performance
4. Display Top 10 Students
5. Exit Program
Enter your choice: 4
Displaying top 10 students after Merge Sort:

--- Top 10 Students ---
1. Student(Name: Student_21, Roll: 21, CGPA: 9.48)
2. Student(Name: Student_13, Roll: 13, CGPA: 9.21)
3. Student(Name: Student_27, Roll: 27, CGPA: 8.98)
4. Student(Name: Student_20, Roll: 20, CGPA: 8.90)
5. Student(Name: Student_5, Roll: 5, CGPA: 8.08)
6. Student(Name: Student_29, Roll: 29, CGPA: 7.62)
7. Student(Name: Student_1, Roll: 1, CGPA: 7.60)
8. Student(Name: Student_11, Roll: 11, CGPA: 7.58)
9. Student(Name: Student_12, Roll: 12, CGPA: 7.04)
10. Student(Name: Student_16, Roll: 16, CGPA: 6.97)
----------------------

--- Student Records Program ---
1. Generate Student Data
2. Sort using Quick Sort and display performance
3. Sort using Merge Sort and display performance
4. Display Top 10 Students
5. Exit Program
Enter your choice:
```

Explanation:

The code stores student details, sorts them by CGPA in descending order using two sorting methods, compares their execution time, and prints the top 10 highest-scoring students.

Task 2: **Implementing Bubble Sort with AI CommentsPrompt:**

Write a Python Bubble Sort program. Add inline comments explaining swapping, passes, and termination, and include a short time complexity analysis.

Code:

```
import random
import time

# 1. Define Student class
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"Student(Name: {self.name}, Roll: {self.roll_number}, CGPA:
{self.cgpa:.2f})"

# 2. Implement function to generate student data
def generate_students(num_students):
    students = []
    for i in range(num_students):
        name = f"Student_{i+1}"
        roll_number = i + 1
        cgpa = round(random.uniform(0.0, 10.0), 2)
        students.append(Student(name, roll_number, cgpa))
    return students

# 3. Implement Bubble Sort (descending by CGPA)
def bubble_sort_students_desc(students):
    n = len(students)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place, so we don't need to check them
        # This optimization reduces unnecessary comparisons in later passes.
        # A pass is a full iteration over the unsorted part of the array.
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            # If the current student's CGPA is less than the next student's CGPA, swap
them
            # Swapping means exchanging the positions of two elements to achieve
descending order.
            if students[j].cgpa < students[j + 1].cgpa:
                students[j], students[j + 1] = students[j + 1], students[j]
    # The outer loop completes 'n' passes (though n-1 passes are sufficient for sorting).
    # After each pass, the largest unsorted element 'bubbles' to its correct position.
    # The algorithm terminates when all elements are in their sorted order.
    return students

# 4. Implement function to measure performance
def measure_performance(sort_func, students_data):
    start_time = time.time()
```

```
    sorted_students = sort_func(students_data.copy()) # Use a copy to avoid modifying
original list
    end_time = time.time()
    return sorted_students, (end_time - start_time) * 1000 # Time in milliseconds

# 5. Implement function to display top 10 students
def display_top_10_students(students):
    if not students:
        print("No students to display.")
        return

    print("\n--- Top 10 Students ---")
    for i, student in enumerate(students[:10]):
        print(f"{i+1}. {student}")
    print("-----------------------")

# 6. Main program loop (User Interface)
def main():
    student_records = None
    sorted_records = None

    while True:
        print("\n--- Bubble Sort for Student Records ---")
        print("1. Generate Student Data")
        print("2. Sort using Bubble Sort and display performance")
        print("3. Display Top 10 Students")
        print("4. Exit Program")
        choice = input("Enter your choice: ")

        if choice == '1':
            try:
                num_students = int(input("Enter the number of students to generate: "))
                if num_students <= 0:
                    raise ValueError
                print(f"Generating {num_students} students...")
                student_records = generate_students(num_students)
                sorted_records = None # Reset sorted data after new generation
                print("Student data generated successfully.")
            except ValueError:
                print("Invalid input. Please enter a positive integer for the number of
students.")
        elif choice == '2':
            if student_records is None:
                print("Please generate student data first (Option 1).")
                continue
            print("Sorting using Bubble Sort...")
            sorted_records, time_taken = measure_performance(bubble_sort_students_desc,
student_records)
            print(f"Bubble Sort completed in {time_taken:.2f} ms.")
        elif choice == '3':
            if sorted_records is None:
                if student_records is None:
                    print("Please generate and then sort student data first (Options 1,
then 2).")
```

```
                else:
                    print("Please sort the student data first (Option 2).")
                continue
            print(f"Displaying top 10 students after Bubble Sort:")
            display_top_10_students(sorted_records)
        elif choice == '4':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 4.")

if __name__ == "__main__":
    main()
```

Output:

```
--- Bubble Sort for Student Records ---
1. Generate Student Data
2. Sort using Bubble Sort and display performance
3. Display Top 10 Students
4. Exit Program
Enter your choice: 1
Enter the number of students to generate: 20
Generating 20 students...
Student data generated successfully.

--- Bubble Sort for Student Records ---
1. Generate Student Data
2. Sort using Bubble Sort and display performance
3. Display Top 10 Students
4. Exit Program
Enter your choice: 3
Please sort the student data first (Option 2).

--- Bubble Sort for Student Records ---
1. Generate Student Data
2. Sort using Bubble Sort and display performance
3. Display Top 10 Students
4. Exit Program
Enter your choice: 4
Exiting program. Goodbye!
(.venv) PS C:\Users\lilac\OneDrive\Desktop\AI-Assisted-Coding>
```

Explanation:

The code sorts elements by repeatedly comparing and swapping adjacent values until the list is sorted. Inline comments explain each pass and how the algorithm stops when no swaps occur.

Task 3: Quick Sort and Merge Sort Comparison

Prompt:

Complete partially written recursive Quick Sort and Merge Sort functions in Python, add proper docstrings, test them on random, sorted, and reverse-sorted lists, and include a short complexity comparison.

Code:

```
import random
import time
import copy # Import copy module for deep copy if needed, though slice is often enough

class Student:
    """
    Represents a student with a name, roll number, and CGPA.
    """
    def __init__(self, name, roll, cgpa):
        """
        Initializes a Student object.

        Args:
            name (str): The name of the student.
            roll (int): The roll number of the student.
            cgpa (float): The CGPA (Cumulative Grade Point Average) of the student.
        """
        self.name = name
        self.roll = roll
        self.cgpa = cgpa

    def __repr__(self):
        """
        Returns a string representation of the Student object.

        Returns:
            str: A string in the format 'Student(Name: ..., Roll: ..., CGPA: ...)'
        """
        return f"Student(Name: {self.name}, Roll: {self.roll}, CGPA: {self.cgpa:.2f})"

# --- List Generation Functions ---
def generate_random_students(num_students):
    """
    Generates a list of Student objects with random CGPA values.

    Args:
        num_students (int): The number of student records to generate.

    Returns:
        list: A list of Student objects with random CGPA values.
    """
    students = []
    for i in range(num_students):
        name = f"Student_{i+1}"
        roll = i + 1
        cgpa = round(random.uniform(5.0, 10.0), 2) # CGPA between 5.0 and 10.0
```

```
        students.append(Student(name, roll, cgpa))
    return students

def generate_descending_students(num_students):
    """
    Generates a list of Student objects with CGPA values in descending order.

    Args:
        num_students (int): The number of student records to generate.

    Returns:
        list: A list of Student objects with CGPA values sorted in descending order.
    """
    students = []
    base_cgpa = 9.9
    for i in range(num_students):
        name = f"Student_Desc_{i+1}"
        roll = i + 1
        cgpa = round(max(0.0, base_cgpa - (i * 0.01)), 2) # Decrement by a small amount
        students.append(Student(name, roll, cgpa))
    return students

def generate_ascending_students(num_students):
    """
    Generates a list of Student objects with CGPA values in ascending order.

    Args:
        num_students (int): The number of student records to generate.

    Returns:
        list: A list of Student objects with CGPA values sorted in ascending order.
    """
    students = []
    base_cgpa = 5.0
    for i in range(num_students):
        name = f"Student_Asc_{i+1}"
        roll = i + 1
        cgpa = round(min(10.0, base_cgpa + (i * 0.01)), 2) # Increment by a small amount
        students.append(Student(name, roll, cgpa))
    return students

# --- Quick Sort Implementation ---
def _partition(arr, low, high):
    """
    Partitions the array around a pivot element.
    Elements with CGPA greater than or equal to the pivot are placed before it,
    and elements with CGPA less than the pivot are placed after it (for descending sort).

    Args:
        arr (list): The list of Student objects to be partitioned.
        low (int): The starting index of the subarray.
        high (int): The ending index of the subarray.

    Returns:
```

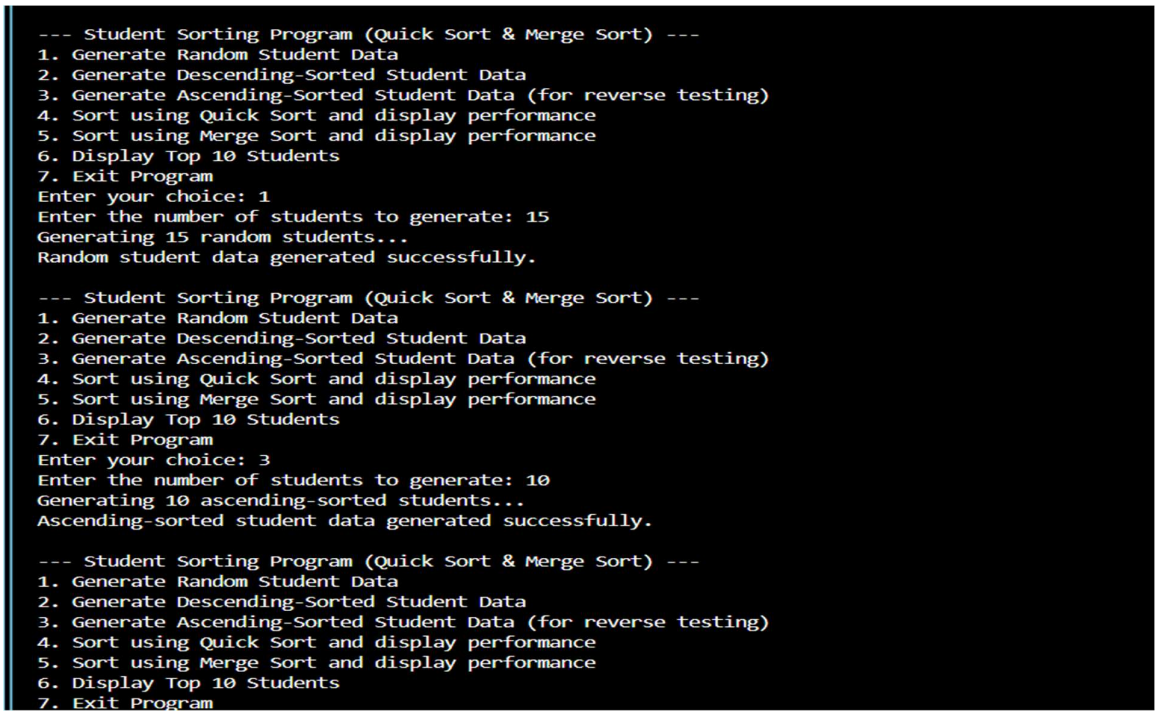
```
        int: The index of the pivot element after partitioning.
    """
    pivot = arr[high].cgpa
    i = low - 1
    for j in range(low, high):
        if arr[j].cgpa >= pivot: # Sort in descending order
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    """
    Sorts a list of Student objects in descending order based on their CGPA
    using the Quick Sort algorithm.

    Args:
        arr (list): The list of Student objects to be sorted.
        low (int): The starting index of the subarray to be sorted.
        high (int): The ending index of the subarray to be sorted.

    Returns:
        list: The sorted list of Student objects (in-place sort, returns the same list).
    """
    if low < high:
        pi = _partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
    return arr

# --- Merge Sort Implementation ---
def merge_sort(arr):
    """
    Sorts a list of Student objects in descending order based on their CGPA
    using the Merge Sort algorithm.

    Args:
        arr (list): The list of Student objects to be sorted.

    Returns:
        list: A new list containing the sorted Student objects.
    """
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        # Merge the two halves back into the original array
        while i < len(left_half) and j < len(right_half):
```

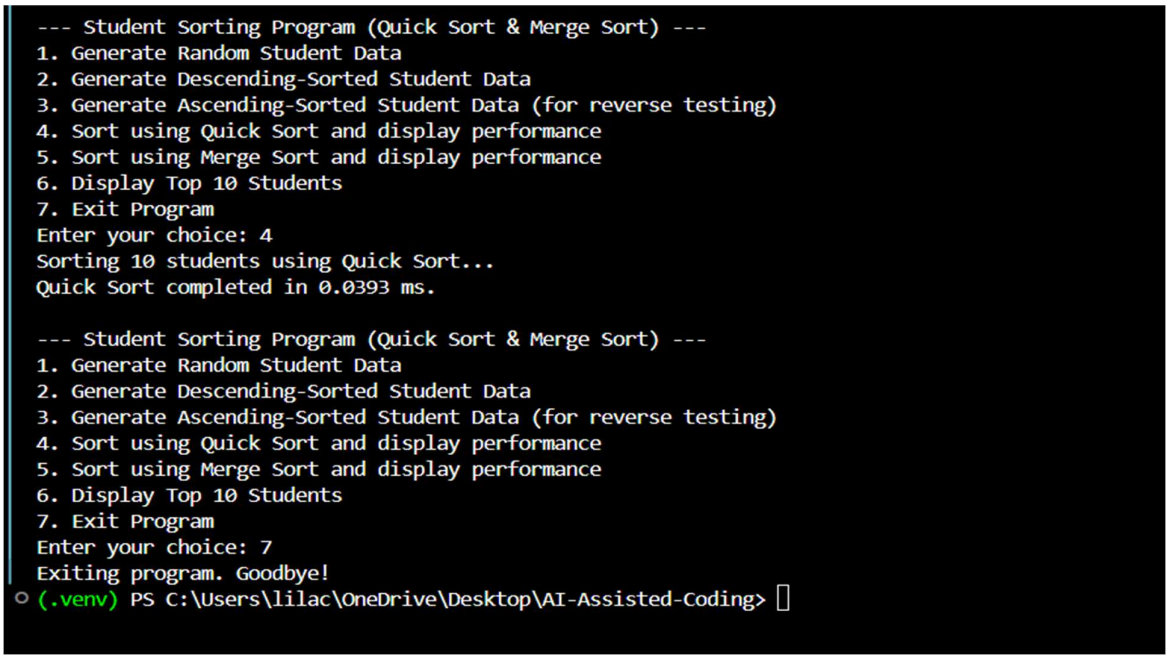
```
            # Sort in descending order based on CGPA
            if left_half[i].cgpa >= right_half[j].cgpa:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Copy any remaining elements of left_half
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        # Copy any remaining elements of right_half
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
    return arr

# --- Performance Measurement and Display Functions ---
def measure_performance(sort_func, students_data, *args):
    """
    Measures the runtime performance of a sorting function.

    Args:
        sort_func (function): The sorting function to measure.
        students_data (list): The list of Student objects to be sorted.
        *args: Additional arguments to pass to the sorting function (e.g., low, high for
Quick Sort).

    Returns:
        tuple: A tuple containing the sorted list and the time taken in milliseconds.
    """
    data_copy = copy.deepcopy(students_data) # Use deepcopy for nested objects like
Student
    start_time = time.time()
    if sort_func.__name__ == 'quick_sort':
        sorted_students = sort_func(data_copy, *args)
    else:
        sorted_students = sort_func(data_copy)
    end_time = time.time()
    return sorted_students, (end_time - start_time) * 1000 # Time in milliseconds

def display_top_10_students(students, sort_method):
    """
    Displays the top 10 students from a sorted list.

    Args:
        students (list): The list of sorted Student objects.
        sort_method (str): The name of the sorting method used.
    """
```

```
    if not students:
        print("No students to display.")
        return

    print(f"\n--- Top 10 Students (after {sort_method}) ---")
    for i, student in enumerate(students[:10]):
        print(f"{i+1}. {student}")
    print("-----------------------")

# --- Main Program Loop (User Interface) ---
def main():
    student_records = None
    sorted_records = None
    current_sort_method = None
    num_students_generated = 0

    while True:
        print("\n--- Student Sorting Program (Quick Sort & Merge Sort) ---")
        print("1. Generate Random Student Data")
        print("2. Generate Descending-Sorted Student Data")
        print("3. Generate Ascending-Sorted Student Data (for reverse testing)")
        print("4. Sort using Quick Sort and display performance")
        print("5. Sort using Merge Sort and display performance")
        print("6. Display Top 10 Students")
        print("7. Exit Program")
        choice = input("Enter your choice: ")

        if choice == '1':
            try:
                num_students = int(input("Enter the number of students to generate: "))
                if num_students <= 0:
                    raise ValueError
                print(f"Generating {num_students} random students...")
                student_records = generate_random_students(num_students)
                num_students_generated = num_students
                sorted_records = None # Reset sorted data after new generation
                current_sort_method = None
                print("Random student data generated successfully.")
            except ValueError:
                print("Invalid input. Please enter a positive integer for the number of
students.")
        elif choice == '2':
            try:
                num_students = int(input("Enter the number of students to generate: "))
                if num_students <= 0:
                    raise ValueError
                print(f"Generating {num_students} descending-sorted students...")
                student_records = generate_descending_students(num_students)
                num_students_generated = num_students
                sorted_records = None
                current_sort_method = None
                print("Descending-sorted student data generated successfully.")
            except ValueError:
```

```
                print("Invalid input. Please enter a positive integer for the number of
students.")
        elif choice == '3':
            try:
                num_students = int(input("Enter the number of students to generate: "))
                if num_students <= 0:
                    raise ValueError
                print(f"Generating {num_students} ascending-sorted students...")
                student_records = generate_ascending_students(num_students)
                num_students_generated = num_students
                sorted_records = None
                current_sort_method = None
                print("Ascending-sorted student data generated successfully.")
            except ValueError:
                print("Invalid input. Please enter a positive integer for the number of
students.")
        elif choice == '4':
            if student_records is None:
                print("Please generate student data first (Option 1, 2, or 3).")
                continue
            print(f"Sorting {num_students_generated} students using Quick Sort...")
            # Quick Sort requires low and high indices
            sorted_records, time_taken = measure_performance(quick_sort, student_records,
0, len(student_records) - 1)
            current_sort_method = "Quick Sort"
            print(f"Quick Sort completed in {time_taken:.4f} ms.")
        elif choice == '5':
            if student_records is None:
                print("Please generate student data first (Option 1, 2, or 3).")
                continue
            print(f"Sorting {num_students_generated} students using Merge Sort...")
            sorted_records, time_taken = measure_performance(merge_sort, student_records)
            current_sort_method = "Merge Sort"
            print(f"Merge Sort completed in {time_taken:.4f} ms.")
        elif choice == '6':
            if sorted_records is None:
                if student_records is None:
                    print("Please generate and then sort student data first (Options
1/2/3, then 4 or 5).")
                else:
                    print("Please sort the student data first (Option 4 or 5).")
                continue
            display_top_10_students(sorted_records, current_sort_method)
        elif choice == '7':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 7.")

if __name__ == "__main__":
    main()
```

Output:





Explanation:

The code completes recursive Quick Sort and Merge Sort functions, tests them on different input orders, and compares their performance behavior in various cases.

Task 4: Real-Time Application – Inventory Management System

Prompt:

Suggest efficient search and sort algorithms for a large inventory system, justify choices based on dataset size and performance, and implement the recommended search and sort functions in Python.

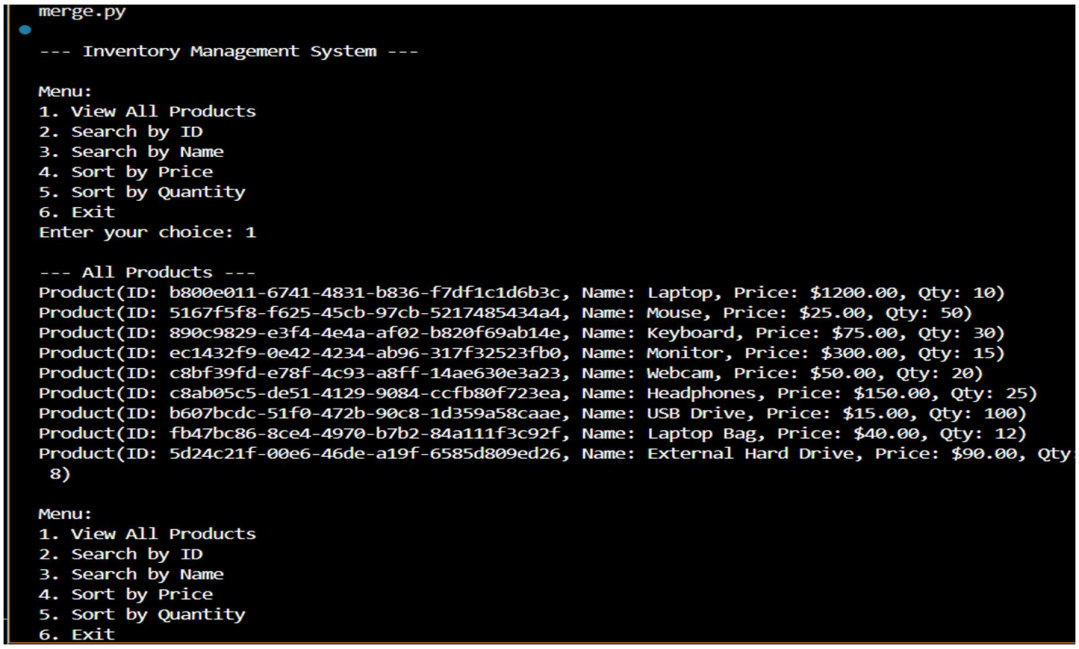
Code:

```
import uuid

class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"Product(ID: {self.product_id}, Name: {self.name}, Price:
${self.price:.2f}, Qty: {self.quantity})"

def generate_sample_inventory():
    products = [
        Product(str(uuid.uuid4()), "Laptop", 1200.00, 10),
        Product(str(uuid.uuid4()), "Mouse", 25.00, 50),
        Product(str(uuid.uuid4()), "Keyboard", 75.00, 30),
        Product(str(uuid.uuid4()), "Monitor", 300.00, 15),
        Product(str(uuid.uuid4()), "Webcam", 50.00, 20),
        Product(str(uuid.uuid4()), "Headphones", 150.00, 25),
        Product(str(uuid.uuid4()), "USB Drive", 15.00, 100),
        Product(str(uuid.uuid4()), "Laptop Bag", 40.00, 12),
        Product(str(uuid.uuid4()), "External Hard Drive", 90.00, 8)
    ]
    return products

def search_by_id(inventory, product_id_to_find):
    id_to_product_map = {product.product_id: product for product in inventory}
    return id_to_product_map.get(product_id_to_find)

def search_by_name(inventory, name_to_find):
    matching_products = []
    name_to_find_lower = name_to_find.lower()
    for product in inventory:
        if name_to_find_lower in product.name.lower():
            matching_products.append(product)
    return matching_products

def sort_inventory_by_price(inventory):
    return sorted(inventory, key=lambda product: product.price)

def sort_inventory_by_quantity(inventory):
    return sorted(inventory, key=lambda product: product.quantity)

def run_inventory_system():
```

```
inventory = generate_sample_inventory()
print("\n--- Inventory Management System ---")

while True:
    print("\nMenu:")
    print("1. View All Products")
    print("2. Search by ID")
    print("3. Search by Name")
    print("4. Sort by Price")
    print("5. Sort by Quantity")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
        if not inventory:
            print("Inventory is empty.")
        else:
            print("\n--- All Products ---")
            for product in inventory:
                print(product)
    elif choice == '2':
        product_id = input("Enter Product ID to search: ")
        found_product = search_by_id(inventory, product_id)
        if found_product:
            print("\nProduct Found:")
            print(found_product)
        else:
            print(f"No product found with ID: {product_id}")
    elif choice == '3':
        product_name = input("Enter Product Name (or part of it) to search: ")
        found_products = search_by_name(inventory, product_name)
        if found_products:
            print("\nMatching Products:")
            for product in found_products:
                print(product)
        else:
            print(f"No products found matching name: {product_name}")
    elif choice == '4':
        sorted_by_price = sort_inventory_by_price(inventory)
        print("\n--- Products Sorted by Price (Ascending) ---")
        for product in sorted_by_price:
            print(product)
    elif choice == '5':
        sorted_by_quantity = sort_inventory_by_quantity(inventory)
        print("\n--- Products Sorted by Quantity (Ascending) ---")
        for product in sorted_by_quantity:
            print(product)
    elif choice == '6':
        print("Exiting Inventory Management System. Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```

```
if __name__ == '__main__':
    run_inventory_system()
```

Output:



Explanation:

The code recommends suitable search and sorting methods for managing large product data, explains why they fit the system needs, and implements functions to perform fast searching and sorting.

Task 5: Real-Time Stock Data Sorting & Searching

Prompt:

Simulate stock data (Symbol, Opening, Closing), calculate percentage change, sort stocks using Heap Sort by daily gain/loss, implement search using a Hash Map, compare with sorted() and dict lookups, and briefly analyze trade-offs.

Code:

```
import random
import string
import time # Import the time module for performance measurement

# 1. Define the Stock class
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price

    def percentage_change(self):
        if self.opening_price == 0:
            return float('inf') if self.closing_price > 0 else 0.0 # Avoid division by
zero
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100

    def __repr__(self):
        return f"Stock(Symbol: {self.symbol}, Open: {self.opening_price:.2f}, Close:
{self.closing_price:.2f}, Change: {self.percentage_change():.2f}%)"

    # For heap sort comparison (descending order by percentage_change) - used for min-heap
to build max-heap behavior
    # For max-heap, we usually compare if parent is smaller than child.
    # Python's heapq is a min-heap, so for custom objects, if we want max-heap behavior,
we often negate the comparison key.
    # However, our heap_sort implementation is custom and builds a max-heap directly by
comparing values.
    # The __lt__ is primarily useful if we were to use Python's `sorted()` or
`list.sort()` directly without a key.
    # For `heap_sort` which directly compares `percentage_change()`, this `__lt__` is not
strictly used by `heapify`.
    # Let's refine `__lt__` to support direct sorting for `sorted()` if needed, but for
heap_sort we rely on direct calls.
    # For `heap_sort` to arrange in descending order, our `heapify` compares
`arr[i].percentage_change() < arr[left].percentage_change()`
    # and then `arr[largest].percentage_change() < arr[right].percentage_change()` which
means it builds a max-heap.
    # The final `arr[::-1]` reverses it to get descending order if the heap was built for
ascending.
    # Let's adjust the `__lt__` to make `sorted()` work for descending easily.
    def __lt__(self, other): # This defines less than. For descending, we want greater to
be less.
        return self.percentage_change() > other.percentage_change() # Invert for
descending order with standard sort
```

```python
# 2. Function to simulate stock data
def simulate_stock_data(num_stocks):
    stocks = []
    for _ in range(num_stocks):
        symbol = ''.join(random.choices(string.ascii_uppercase, k=3))
        opening_price = round(random.uniform(10.0, 1000.0), 2)
        # Closing price within +/- 10% of opening price
        closing_price = round(opening_price * random.uniform(0.9, 1.1), 2)
        stocks.append(Stock(symbol, opening_price, closing_price))
    return stocks


# 3. Implement Heap Sort
def heapify(arr, n, i):
    largest = i  # Initialize largest as root
    left = 2 * i + 1
    right = 2 * i + 2

    # See if left child of root exists and is greater than root (by percentage_change)
    if left < n and arr[left].percentage_change() > arr[largest].percentage_change():
        largest = left

    # See if right child of root exists and is greater than current largest
    if right < n and arr[right].percentage_change() > arr[largest].percentage_change():
        largest = right

    # Change root, if needed
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]  # swap
        heapify(arr, n, largest) # Heapify the root.

def heap_sort(arr):
    n = len(arr)
    # Build a maxheap (rearrange array) to sort in descending order
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # One by one extract elements from maxheap
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]  # swap root (largest) with last element
        heapify(arr, i, 0)  # Call heapify on the reduced heap

    # The heapify process places largest elements at the end, resulting in ascending order
    # for the `arr` itself after the loop. To get descending order of percentage change,
    # we need to reverse the result. Or, the heapify should be adapted to build a min-heap
    # and extract elements, or make comparison inverse.
    # Given `heapify` creates a max-heap (largest at root) and then puts largest at end,
    # the array `arr` will be sorted in ASCENDING order after the loop.
    # So, we reverse it to get DESCENDING order of percentage change.
    return arr[::-1]
```

```python
# 4. Create a function to build a hash map
def build_hash_map(stocks):
    stock_map = {}
    for stock in stocks:
        stock_map[stock.symbol] = stock
    return stock_map


# 5. Implement a function for hash map lookup
def hash_map_lookup(hash_map, symbol):
    return hash_map.get(symbol)


# New: Linear Search Function
def linear_search(stocks, symbol):
    for stock in stocks:
        if stock.symbol == symbol:
            return stock
    return None

# New: Performance Measurement Functions
def measure_sorting_performance(stocks_to_sort):
    print("\n--- Measuring Sorting Performance ---")
    if not stocks_to_sort:
        print("No stock data to sort. Please generate data first.")
        return

    # Heap Sort
    start_time = time.perf_counter()
    heap_sorted_stocks = heap_sort(list(stocks_to_sort)) # Pass a copy
    end_time = time.perf_counter()
    heap_sort_time = (end_time - start_time) * 1000
    print(f"Heap Sort time: {heap_sort_time:.4f} ms")
    # print(f"First 5 Heap Sorted: {heap_sorted_stocks[:5]}") # Optional: print a few to
verify

    # Python's built-in sorted()
    start_time = time.perf_counter()
    # Use key for sorting by percentage_change in descending order
    builtin_sorted_stocks = sorted(list(stocks_to_sort), key=lambda stock:
stock.percentage_change(), reverse=True)
    end_time = time.perf_counter()
    builtin_sort_time = (end_time - start_time) * 1000
    print(f"Built-in sorted() time: {builtin_sort_time:.4f} ms")
    # print(f"First 5 Built-in Sorted: {builtin_sorted_stocks[:5]}") # Optional: print a
few to verify

    print("-------------------------------------")

def measure_search_performance(stocks, stock_map):
    print("\n--- Measuring Search Performance ---")
    if not stocks:
        print("No stock data to search. Please generate data first.")
        return
```

```
    # Select a random symbol to search for, ensuring it exists for fair comparison
    search_symbol_exists = random.choice(stocks).symbol
    search_symbol_non_existent = ''.join(random.choices(string.ascii_uppercase, k=3)) +
'X'
    # Ensure non_existent is truly non-existent
    while search_symbol_non_existent in stock_map:
        search_symbol_non_existent = ''.join(random.choices(string.ascii_uppercase, k=3))
+ 'X'

    # --- Existing Stock Search ---
    print(f"Searching for an existing symbol: {search_symbol_exists}")

    # Hash Map Lookup
    start_time = time.perf_counter()
    found_by_hash = hash_map_lookup(stock_map, search_symbol_exists)
    end_time = time.perf_counter()
    hash_map_time = (end_time - start_time) * 1000
    print(f"Hash Map lookup time (found): {hash_map_time:.6f} ms")

    # Linear Search
    start_time = time.perf_counter()
    found_by_linear = linear_search(stocks, search_symbol_exists)
    end_time = time.perf_counter()
    linear_search_time = (end_time - start_time) * 1000
    print(f"Linear Search time (found): {linear_search_time:.6f} ms")

    # --- Non-Existing Stock Search ---
    print(f"\nSearching for a non-existent symbol: {search_symbol_non_existent}")

    # Hash Map Lookup
    start_time = time.perf_counter()
    not_found_by_hash = hash_map_lookup(stock_map, search_symbol_non_existent)
    end_time = time.perf_counter()
    hash_map_time_nf = (end_time - start_time) * 1000
    print(f"Hash Map lookup time (not found): {hash_map_time_nf:.6f} ms")

    # Linear Search
    start_time = time.perf_counter()
    not_found_by_linear = linear_search(stocks, search_symbol_non_existent)
    end_time = time.perf_counter()
    linear_search_time_nf = (end_time - start_time) * 1000
    print(f"Linear Search time (not found): {linear_search_time_nf:.6f} ms")

    print("---------------------------------")


# Interactive Command-Line Interface

simulated_stocks = []
stock_lookup_map = {}

def display_menu():
    print("\n--- Stock Analysis Tool CLI ---")
```

```
    print("1. Generate new stock data")
    print("2. Display current stock data")
    print("3. Sort and display stocks by percentage change (Heap Sort)")
    print("4. Search for a stock by symbol")
    print("5. Compare sorting algorithm performance")
    print("6. Compare search algorithm performance")
    print("7. Exit")
    print("------------------------------")

while True:
    display_menu()
    choice = input("Enter your choice (1-7): ")

    if choice == '1':
        try:
            num_stocks = int(input("Enter the number of stocks to simulate: "))
            if num_stocks <= 0:
                print("Please enter a positive number.")
                continue
            simulated_stocks = simulate_stock_data(num_stocks)
            stock_lookup_map = build_hash_map(simulated_stocks) # Rebuild map for new data
            print(f"Generated {num_stocks} stocks.")
        except ValueError:
            print("Invalid input. Please enter a number.")

    elif choice == '2':
        if simulated_stocks:
            print("\n--- Current Stock Data ---")
            for stock in simulated_stocks:
                print(stock)
        else:
            print("No stock data available. Please generate data first (Option 1).")

    elif choice == '3':
        if simulated_stocks:
            print("\n--- Stocks Sorted by Percentage Change (Descending using Heap Sort) -
--")
            # Heap sort modifies the list in place, so we pass a copy for consistent
behavior.
            # The heap_sort function returns a reversed list, which is the desired
descending order.
            sorted_stocks_heap = heap_sort(list(simulated_stocks))
            for stock in sorted_stocks_heap:
                print(stock)
        else:
            print("No stock data available to sort. Please generate data first (Option
1).")

    elif choice == '4':
        if simulated_stocks and stock_lookup_map:
            symbol = input("Enter stock symbol to search: ").upper()
            found_stock = hash_map_lookup(stock_lookup_map, symbol)
            if found_stock:
                print(f"Found stock: {found_stock}")
```

```
            else:
                print(f"Stock with symbol '{symbol}' not found.")
        else:
            print("No stock data available for search. Please generate data first (Option
1).")

    elif choice == '5':
        measure_sorting_performance(simulated_stocks)

    elif choice == '6':
        measure_search_performance(simulated_stocks, stock_lookup_map)

    elif choice == '7':
        print("Exiting Stock Analysis Tool. Goodbye!")
        break

    else:
        print("Invalid choice. Please enter a number between 1 and 7.")
```

Output:

```
--- Stock Analysis Tool CLI ---
1. Generate new stock data
2. Display current stock data
3. Sort and display stocks by percentage change (Heap Sort)
4. Search for a stock by symbol
5. Compare sorting algorithm performance
6. Compare search algorithm performance
7. Exit
------------------------------
Enter your choice (1-7): 1
Enter the number of stocks to simulate: 20
Generated 20 stocks.

--- Stock Analysis Tool CLI ---
1. Generate new stock data
2. Display current stock data
3. Sort and display stocks by percentage change (Heap Sort)
4. Search for a stock by symbol
5. Compare sorting algorithm performance
6. Compare search algorithm performance
7. Exit
------------------------------
Enter your choice (1-7): 3

--- Stocks Sorted by Percentage Change (Descending using Heap Sort) ---
Stock(Symbol: UFL, Open: 596.04, Close: 655.11, Change: 9.91%)
Stock(Symbol: WRG, Open: 633.88, Close: 694.06, Change: 9.49%)
Stock(Symbol: YOQ, Open: 68.78, Close: 75.04, Change: 9.10%)
Stock(Symbol: JKD, Open: 352.07, Close: 382.82, Change: 8.73%)
Stock(Symbol: WVD, Open: 863.04, Close: 934.50, Change: 8.28%)
```

```
Stock(Symbol: AXY, Open: 994.36, Close: 962.21, Change: -3.23%)
Stock(Symbol: NET, Open: 679.99, Close: 653.40, Change: -3.91%)
Stock(Symbol: ZCT, Open: 791.65, Close: 749.31, Change: -5.35%)
Stock(Symbol: TDZ, Open: 118.47, Close: 111.14, Change: -6.19%)
Stock(Symbol: NDG, Open: 621.43, Close: 572.26, Change: -7.91%)
Stock(Symbol: LDC, Open: 162.47, Close: 149.20, Change: -8.17%)
Stock(Symbol: USL, Open: 815.95, Close: 748.71, Change: -8.24%)
Stock(Symbol: AUM, Open: 486.99, Close: 441.18, Change: -9.41%)

--- Stock Analysis Tool CLI ---
1. Generate new stock data
2. Display current stock data
3. Sort and display stocks by percentage change (Heap Sort)
4. Search for a stock by symbol
5. Compare sorting algorithm performance
6. Compare search algorithm performance
7. Exit
------------------------------
Enter your choice (1-7): 5

--- Measuring Sorting Performance ---
Heap Sort time: 0.1320 ms
Built-in sorted() time: 0.0250 ms
----------------------------------

--- Stock Analysis Tool CLI ---
1. Generate new stock data
2. Display current stock data
3. Sort and display stocks by percentage change (Heap Sort)
4. Search for a stock by symbol
5. Compare sorting algorithm performance
6. Compare search algorithm performance
7. Exit
```

Explanation:

The code generates stock data, ranks stocks based on percentage change using Heap Sort, and retrieves stock details quickly using a hash map, then compares custom methods with built-in Python functions.